MCP Servers vs. Skills: How to Add Capabilities to an LLM

If you're looking to add capabilities to an LLM, look no further than **MCP servers and skills**.

While an LLM is very general-purpose and knows a lot of things, these two concepts can help you add custom and unique data or capabilities to your LLM for all kinds of use cases.

This could include:

- Coding assistance
- Adding extra information and data
- Building AI agents
- Many other use cases

You're going to learn whether to choose **MCP**, **skills**, or **both**.

The LLM as a Prediction Machine

Think of an LLM as essentially a crystal-ball prediction machine that has been trained on all types of information: books, magazines, and maybe even some of your internet threads.

It can use this information, along with pattern recognition, to answer many different questions.

For example:

- What's the history of Red Hat?
- How do we inspect a specific database?

The LLM may already know a lot of information and may even know how to work with your specific database.

But getting the right answer means providing the **right context** to the LLM.

That context might include additional information such as:

- The way we want our data formatted
- The way our database is configured for our specific team
- A tool response that retrieves information from the database and returns it to the LLM's context window

When we're simply asking a question with a specific role and task, that's known as **prompt engineering**.

We're giving a task to the LLM.

But when we add all of this additional information, things get more interesting.

This is not just prompt engineering.

This is known as **context engineering**.

Context engineering is about giving the right context to a model so it can produce the right answer.

We're giving the model the information it needs.

The big question is:

How do we give an AI model the correct context to produce the right answer?

And:

How do we take this and build an AI agent?

Let's get into it.

MCP: Connecting an LLM to External Systems

Let's say an agent needs data that is currently stored in a **Customer Relationship Management system**, or CRM.

Think about the LLM we're using to build the agent.

Instead of copying and pasting the CRM service's API documentation, giving the LLM a token, and saying:

> Please update our customer's contact information, make no mistakes.

...and then crossing our fingers, we can use **MCP**, or the **Model Context Protocol**.

MCP standardizes how an AI model communicates with different data sources.

It abstracts service APIs into a simpler, LLM-ready format.

It can also handle authentication, which is extremely important when an agent needs to make calls to an external service.

This can provide a uniquely scoped token with:

- Read access
- Write access

- Whatever permissions are required

Behind the scenes, the MCP server is added to your IDE or AI application.

It tells the LLM how to provide specific structured requests, such as JSON requests, for the information it needs from the service.

The MCP server then translates those requests into things such as:

- POST requests
- GET requests

Those requests are then sent to the service itself.

MCP is the standardized layer between the LLM and the external data sources or services it needs to access.

It is supported by many AI tools.

The Missing Piece: Domain Knowledge and Repeatability

MCP solves the problem of:

How do we give external data to an LLM?

But there's still another problem:

How do we give the LLM domain knowledge or procedures that it might not already have?

With MCP, we can pull customer records from a CRM.

But perhaps our sales team wants the information handled in the exact same way every time.

If you know anything about LLMs, you know they are non-deterministic.

That can make consistency difficult.

For example, perhaps the sales team wants CRM data formatted with:

- The customer's name
- The customer's contact information
- Their favorite type of cookie

Being able to create a repeatable way to say:

> I want this formatted the same way every time.

...is the basis of why **skills** are so important.

Skills: Teaching the Model How to Do Something

Think about tasks you repeatedly use an LLM for, such as:

- Cleaning up Excel documents
- Debugging code
- Running verification procedures
- Performing compliance checks

The prompts and scripts you use for these tasks can be packaged into a **skill**.

A skill is essentially a Markdown file with some additional metadata, packaged inside a folder.

A skill might contain:

- A title identifying the skill
- A description explaining when the skill should be used
- The actual instructions or prompt that will be passed to the LLM

What's especially useful is that a skill can be **automatically loaded into the LLM's context window when needed**.

For example, suppose you have a **code debugger skill**.

That skill only needs to be loaded when you're asking about code errors.

The skill folder can also contain:

- Additional resources
- Reference material
- Examples
- Scripts

When additional context or capabilities are needed, those resources can be included with the skill and loaded when the model needs that specific capability.

MCP vs. Skills: When Should You Use Each?

Use MCP for External Data and Tool Access

When your AI application needs access to **real-time data** in a controlled and tightly permissioned way, MCP is usually the right choice.

Think of MCP as an integration layer between:

Agents and tools

The agent can call the specific tools and resources that the MCP server provides.

Examples include:

- What virtual machines am I currently running?
- What is the current state of our cluster?
- What information do we have about a particular customer?

MCP is useful when the AI needs to interact with external systems or retrieve current information.

However, setting up and configuring an MCP server can be overkill when you simply need to add a reusable custom capability to your AI.

That's where skills shine.

Use Skills for Reusable Knowledge and Procedures

Skills are lightweight.

They teach your model **how to do something**.

For example, a skill might teach the model how to:

- Fetch investment data and analyze it
- Debug code using a specific methodology
- Follow a company-specific compliance process
- Format a report consistently
- Use included scripts and examples to perform a task

Both MCP servers and skills can enhance the context available to an LLM.

That additional context can help the model produce the correct answer or output when you're building or using AI agents.

The Simple Mental Model

Think about the difference this way:

MCP gives the AI access to things.

Skills teach the AI how to do things.

An MCP server might let an agent retrieve customer information from a CRM.

A skill might tell the agent exactly how your company wants that customer information analyzed, formatted, validated, and presented.

In many real-world AI systems, the answer isn't MCP **or** skills.

It's **MCP plus skills**.

MCP provides access to the external systems and data.

Skills provide the reusable procedures, domain knowledge, and workflows for using that information correctly.

Both can be used locally on your machine and integrated into modern AI and agent workflows.

Another related topic is using **command-line interfaces, or CLIs, to build agents**.

That's another powerful area for extending agentic systems and combining tools, context, and reusable workflows.